

Voice-to-Form Health Clinic

Complete Azure Project Explanation and Technical Report

Scalable Voice-to-Form Translator - Azure Only

Solution scenario	Health clinic patient intake
Primary Azure region	Canada Central
Disaster recovery region	Canada East
Public URL	https://voicetoform.amtcloud.site
Availability model	Traffic Manager priority routing - Central primary, East failover
Report status	Final implementation summary - August 2026

Report purpose
This document explains the project from the beginning: the problem being solved, the Azure architecture, the resources created, the implementation sequence, application code flow, security, high availability, monitoring, testing, major problems encountered, fixes applied, and the final operating model.

Contents

1. Executive summary
 2. Business problem and project objective
 3. Final solution architecture
 4. Azure resource inventory
 5. Phase 1 - Planning, governance, and regional design
 6. Phase 2 - Storage, security, encryption, and managed identities
 7. Phase 3 - AI services and document-processing foundation
 8. Phase 4 - Web App and patient-facing frontend
 9. Phase 5 - Azure Functions and asynchronous processing
 10. Phase 6 - API Management and Application Gateway
 11. Phase 7 - Custom domain, HTTPS, and certificates
 12. Phase 8 - Microsoft Entra External ID authentication
 13. Phase 9 - End-to-end Voice-to-Form processing
 14. Phase 10 - Canada East disaster recovery build
 15. Phase 11 - Traffic Manager failover and failback
 16. Monitoring and observability
 17. Major technical problems and how they were solved
 18. Security architecture
 19. Testing and evidence
 20. Current operating procedure and cost hygiene
 21. Lessons learned and future improvements
 22. Conclusion
- Appendix A - Important resource names
- Appendix B - Useful validation commands
- Appendix C - Screenshot evidence checklist

1. Executive summary

The Voice-to-Form Health Clinic project is an Azure-based patient intake solution that allows a user to choose a language, record a spoken health statement, submit the audio securely, and receive a completed PDF after the backend processes the recording. The solution combines a Node.js web application, Azure Storage, Azure Functions, Azure AI services, API Management, Application Gateway with WAF, Microsoft Entra External ID, Key Vault, monitoring services, and a two-region disaster recovery design.

The project was built around four main goals:

- Usability - patients can provide information by voice instead of typing a long form.
- Security - HTTPS, WAF, authentication, managed identities, Key Vault, and storage encryption protect the solution.
- Scalability - audio processing is decoupled from the web request through Azure Queue Storage and Azure Functions.
- Resilience - Canada Central is the primary region and Canada East is the failover region, with Azure Traffic Manager providing DNS-based priority routing.

Final result

The custom domain is live, the frontend records and submits voice, storage and queue processing work, the Central QueueWorker completes submissions and generates PDFs, authentication works, and Traffic Manager failover/failback between the two Application Gateways has been proven.

2. Business problem and project objective

Traditional patient intake forms can be difficult for patients who are not comfortable typing, who speak another language, or who need a faster way to provide information. The project therefore treats voice as the input channel and converts it into a structured digital workflow.

2.1 Intended patient journey

1. The patient visits the clinic website through the custom domain.
2. The patient authenticates through Microsoft Entra External ID.
3. The patient selects a language and starts the voice intake process.
4. The browser records the patient statement.
5. The web application submits the audio and creates a submission record.
6. The audio is stored temporarily and a queue message is created.
7. The backend function processes the queued work asynchronously.
8. Speech-to-Text converts the recording into text.
9. Translator converts the transcript to English when required.
10. The submission is finalized and a PDF is written to storage.
11. The frontend polls submission status and offers the completed PDF to the patient.

2.2 Why asynchronous processing was chosen

Speech recognition, translation, document handling, and PDF generation can take longer than a normal web request. Instead of making the browser wait for all processing to finish, the Web App stores the audio, places a message on Azure Queue Storage, and immediately returns a submission identifier. The frontend

then checks the submission status until it changes to completed. This design is more scalable and more fault tolerant than doing all work inside one HTTP request.

3. Final solution architecture

The final architecture uses a public custom domain in front of Azure Traffic Manager. Traffic Manager sends normal traffic to the Canada Central Application Gateway and redirects traffic to Canada East when the primary endpoint is unavailable. Both regional paths use Application Gateway and API Management before reaching the regional App Service. The processing pipeline uses shared storage and a queue-driven Function workflow.

Voice-to-Form Health Clinic - Azure Architecture

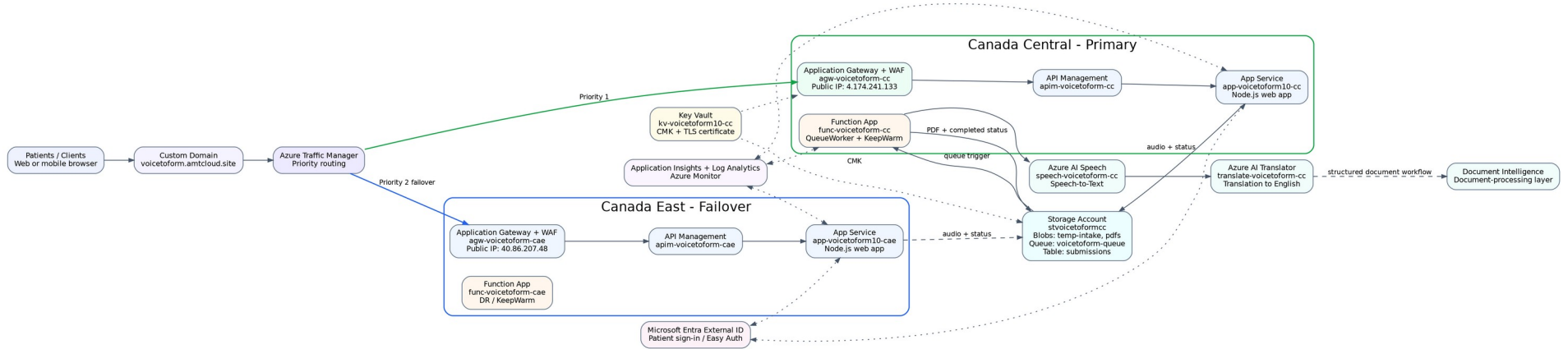


Figure 1. Final Voice-to-Form Azure architecture (logical view).

3.1 Primary traffic path

12. User opens <https://voicetoform.amtcloud.site>.
13. DNS resolves the custom domain to the Traffic Manager profile.
14. Traffic Manager selects the Priority 1 Central endpoint when it is healthy.
15. Traffic reaches the Canada Central Application Gateway over HTTPS.
16. Application Gateway forwards the request toward API Management.
17. API Management routes the request to the Node.js App Service.
18. The Web App handles the patient interface and API endpoints.
19. Audio and submission data are written to Azure Storage and queued for processing.
20. Azure Functions process the queue and use Azure AI services.
21. The completed PDF and final status are written back to Storage.

3.2 Failover traffic path

If the Central Traffic Manager endpoint is disabled or becomes unhealthy, Traffic Manager returns the Canada East Application Gateway address. The East path then follows Application Gateway -> API Management -> East App Service. The East Web App uses managed identity permissions to access the shared storage account. During normal shared-queue operation, the Central QueueWorker is the validated processor; the East QueueWorker can remain disabled until a deliberate DR processing strategy is required.

4. Azure resource inventory

Layer	Azure service	Resource / role	Purpose
Global / DNS	Traffic Manager	tm-voicetoform10	Priority routing across Central and East
Global / DNS	Custom domain	voicetoform.amtcloud.site	Stable public hostname for users
Central edge	Application Gateway + WAF	agw-voicetoform-cc	HTTPS ingress and web application firewall
Central API	API Management	apim-voicetoform-cc	Managed API entry point and policy layer
Central compute	App Service	app-voicetoform10-cc	Node.js patient intake web app
Central compute	Function App	func-voicetoform-cc	QueueWorker and KeepWarm processing
Central data	Storage Account	stvoicetoformcc	Audio blobs, PDFs, queue, status table
Central security	Key Vault	kv-voicetoform10-cc	CMK, certificate and secret protection
Central AI	Azure AI Speech	speech-voicetoform-cc	Speech-to-Text
Central AI	Azure AI Translator	translate-voicetoform-cc	Translation to English
Central AI	Document Intelligence	Document-processing resource	Document extraction/analysis layer in project design
East edge	Application Gateway + WAF	agw-voicetoform-cae	Failover ingress; public IP

			40.86.207.48
East API	API Management	apim-voicetoform-cae	Failover API gateway
East compute	App Service	app-voicetoform10-cae	Failover Node.js web app
East compute	Function App	func-voicetoform-cae	DR function host / KeepWarm
Monitoring	Log Analytics	law-voicetoform-cae	Centralized log workspace for East monitoring
Monitoring	Application Insights	East App Insights resource	Request, failure and performance telemetry
Identity	Microsoft Entra External ID	voiceformclinic tenant	Patient sign-up/sign-in and Easy Auth

5. Phase 1 - Planning, governance, and regional design

The project began by defining the health-sector use case and choosing Azure as the only cloud platform. The primary region was Canada Central to keep the main workload close to the expected Canadian user base. A Canada East recovery region was added later to demonstrate regional resiliency.

5.1 Naming and resource groups

The implementation uses consistent naming to identify region and service. Primary resources generally end in -cc, while disaster recovery resources end in -cae. The main resource groups are rg-voicetoform10-cc and rg-voicetoform10-cae.

5.2 Subscription and policy decisions

During the build, subscription/governance settings were adjusted so the required Azure regions and services could be deployed. Location policy was kept as the main governance restriction instead of using policies that would block required resources. Cost was also considered throughout the build, especially for Application Gateway, API Management, and continuously running compute resources.

6. Phase 2 - Storage, security, encryption, and managed identities

Azure Storage became the shared persistence layer for the solution. The project separated data by purpose instead of placing everything in one container.

6.1 Storage layout

- temp-intake blob container - temporary uploaded patient audio.
- pdfs blob container - completed PDF output.
- voicetoform-queue - work queue that triggers background processing.
- voicetoform-queue-poison - failure queue for messages that exceed retry limits.
- submissions table - tracks each submission and processing status.

6.2 Customer-managed key and Key Vault

A Key Vault and customer-managed key were configured for storage encryption. The key name used during the project was cmk-blob. This added a stronger security model than relying only on Microsoft-managed encryption keys and demonstrated key management as part of the project.

6.3 Managed identities and RBAC

Managed identities were used so Azure resources could access other Azure services without embedding storage account keys in application code. The East App Service system-assigned identity was granted Storage Blob Data Contributor, Storage Queue Data Contributor, and Storage Table Data Contributor on stvoicetoformcc. This exact RBAC step became important during failover testing because the East Web App initially received 403 AuthorizationFailure responses until the roles were assigned.

7. Phase 3 - AI services and document-processing foundation

The project provisioned Azure AI services to turn a spoken intake into usable text and structured output. The core validated AI sequence is Speech-to-Text followed by translation when needed. Document Intelligence was also part of the document-processing design and its endpoint/configuration was reviewed during the project.

- Azure AI Speech receives audio and produces a transcript.
- Azure AI Translator converts non-English text to English.
- Document-processing logic prepares structured information for the final intake document.
- The final backend workflow generates a PDF and saves it in the pdfs container.

Design principle

AI services are called from backend processing, not directly from the browser. This keeps credentials and service integration away from the client-side code.

8. Phase 4 - Web App and patient-facing frontend

The patient-facing application was built as a Node.js web project and deployed to Azure App Service. Local development was performed in Visual Studio Code and the application was first tested on port 8080 before deployment.

8.1 Web project structure

```
VOICETOFORM_WebApp
|-- public
|   |-- index.html
|   |-- styles.css
|   `-- app.js
|-- server.js
|-- package.json
`-- .gitignore
```

8.2 Frontend responsibilities

- Display the Voice-to-Form Health Clinic welcome page.

- Allow the patient to choose a language.
- Use the browser microphone to record audio.
- Submit the audio to the backend.
- Display processing progress.
- Poll the status endpoint using the submission ID.
- Show the View completed PDF button when processing is finished.

8.3 Important backend routes

Route	Purpose
POST /api/submit	Accepts the voice recording, stores it, creates queue work and returns a submission ID.
GET /api/status/:submissionId	Returns current processing state such as processing or completed.
GET /api/download/:submissionId	Returns or redirects to the generated PDF for the completed submission.
GET /api/health	Returns HTTP 200 with {"status":"ok"} when application access is allowed; used as a health route.

9. Phase 5 - Azure Functions and asynchronous processing

The backend processing project was built separately from the web project. Azure Functions receives work from the storage queue so the web server does not need to remain busy while AI processing and PDF generation run.

9.1 Function project

```
VoiceToForm-Functions
|-- host.json
|-- package.json
`-- src/functions
    |-- QueueWorker.js
    `-- KeepWarm.js
```

9.2 Queue trigger configuration

- Queue name: voicetoform-queue
- Connection setting: VoicetoformData
- Worker function: QueueWorker
- KeepWarm function: keeps the host active / provides a simple scheduled execution path.

A critical debugging issue was the exact queue name. At one point the East web application tried to send to voiceform-queue, while the real queue was voicetoform-queue. The result was QueueNotFound. Correcting the name and redeploying fixed the submission path.

10. Phase 6 - API Management and Application Gateway

10.1 API Management

API Management was placed in front of the App Service to provide an API control layer. This makes it possible to centralize routing, policies, throttling, transformation, security rules, and monitoring instead of exposing the application backend as the only API boundary.

10.2 Application Gateway with WAF

Application Gateway provides the public regional entry point. It terminates HTTPS, applies Web Application Firewall protection, and forwards traffic to API Management. The Central gateway uses public IP 4.174.241.133. The East gateway uses public IP 40.86.207.48.

10.3 East backend health configuration

The East gateway originally reported an unhealthy APIM backend because the health probe received HTTP 401. The backend was actually reachable; the 401 was produced by authentication. The custom Application Gateway probe was therefore configured to treat 401 as an acceptable response in addition to 200-399. In the portal field the accepted format was 200-399,401. After the probe was attached to https-settings-apim-cae, backend health changed to Healthy.

Important configuration

The East Application Gateway backend setting must keep hostname override enabled and use Pick host name from backend target. APIM expects its own hostname. Turning hostname override off can break the APIM path.

11. Phase 7 - Custom domain, HTTPS, and certificates

The public domain voicetoform.amtcloud.site was purchased and configured in DNS. The project moved from direct azurewebsites.net access to a stable custom hostname that remains the same whether Traffic Manager sends the user to Central or East.

11.1 DNS records

- CNAME: voicetoform -> tm-voicetoform10.trafficmanager.net
- TXT: asuid.voicetoform retained for App Service custom-domain validation.
- ACME challenge TXT record retained for certificate validation/renewal when required.

11.2 TLS certificate

The first gateway certificate was self-signed and used the wrong hostname, so the browser could not provide a clean trusted experience. A trusted Let's Encrypt certificate for the exact hostname voicetoform.amtcloud.site was later generated and imported. The Central listener was switched to a Key Vault-backed certificate and the East listener used an uploaded PFX certificate.

A future operations item remains: the local win-acme scheduled renewal did not fully automate because of local administrator/task limitations. The certificate renewal process should be documented and tested before expiry so the public HTTPS endpoint remains trusted.

12. Phase 8 - Microsoft Entra External ID authentication

Patient authentication was implemented using Microsoft Entra External ID and App Service Authentication (Easy Auth). The same application registration is used by the Central and East web applications, with redirect URIs for both App Service hostnames and the public custom domain.

12.1 Authentication behavior

- Authentication required for the application.
- Unauthenticated requests use an HTTP 302 redirect to Microsoft sign-in.
- Token store enabled.
- External ID tenant handles patient sign-up/sign-in.
- Email-based authentication and Conditional Access were configured as part of the project.

12.2 Custom-domain redirect fix

During East failover, authentication initially returned the browser to the direct East azurewebsites.net hostname instead of keeping voicetoform.amtcloud.site. The fix was to configure App Service auth forward-proxy behavior so Easy Auth trusts the original host header supplied by Application Gateway:

```
az webapp auth update \
  --resource-group rg-voicetoform10-cae \
  --name app-voicetoform10-cae \
  --proxy-convention Custom \
  --custom-host-header X-Original-Host
```

After this change, a failover test showed that the address bar remained on https://voicetoform.amtcloud.site after sign-in. This was an important final fix because the custom hostname must remain stable regardless of which region is serving the request.

Security note

Client secrets, access tokens, storage keys, connection strings, and PFX passwords should never be included in screenshots or reports. A client secret was exposed during testing and should be treated as rotated/replaced.

13. Phase 9 - End-to-end Voice-to-Form processing

Once the frontend, storage, queue, Function App, and AI services were connected, the project was tested as one end-to-end workflow.

13.1 Detailed runtime sequence

22. The patient opens the custom domain and signs in.
23. The patient selects a language.
24. The browser records the patient voice statement.
25. The frontend sends the recording to POST /api/submit.
26. The Web App stores the audio in temp-intake.
27. The Web App inserts or updates the submission record in the submissions table.
28. The Web App sends a message to voicetoform-queue.
29. QueueWorker receives the queue message.

30. The worker updates the submission to processing.
31. Speech-to-Text creates the original transcript.
32. Translator produces the English translation when required.
33. The worker generates the completed patient intake PDF.
34. The PDF is stored in the pdfs container.
35. The worker changes the table status to completed.
36. The browser polling route detects completion.
37. The interface shows View completed PDF.
38. The patient opens/downloads the completed PDF.

Validated result

A French test submission reached the completed state and produced a PDF. Storage, queue processing, status polling, and PDF delivery were therefore proven together.

14. Phase 10 - Canada East disaster recovery build

After the Central application path was working, the project added a second regional web path in Canada East. The goal was not to create an entirely separate dataset, but to demonstrate that the public application can fail over to a second regional frontend/API stack while continuing to use the shared backend data layer.

- Created rg-voicetoform10-cae.
- Created app-voicetoform10-cae and deployed the same Node.js web application.
- Created func-voicetoform-cae for disaster recovery processing capability.
- Created apim-voicetoform-cae.
- Created agw-voicetoform-cae and WAF/HTTPS listener configuration.
- Created/linked monitoring with law-voicetoform-cae and Application Insights.
- Configured Easy Auth using the same External ID application registration.
- Enabled the East App Service managed identity and granted the required Storage data roles.

14.1 Deployment mistake that was corrected

The Function project was accidentally deployed to the East Web App once. Its package.json attempted to run func start, which is not available as the App Service startup command, producing sh: 1: func: not found. The correct VOICETOFORM_WebApp project was then deployed and the logs showed node server.js and the application listening on port 8080.

15. Phase 11 - Traffic Manager failover and failback

Azure Traffic Manager was added after both regional ingress paths were available. Traffic Manager works at the DNS layer and uses priority routing in this project.

15.1 Traffic Manager configuration

- Profile: tm-voicetoform10
- Routing method: Priority
- Central endpoint: central-agw -> 4.174.241.133 -> Priority 1
- East endpoint: east-agw -> 40.86.207.48 -> Priority 2

- Monitor protocol: HTTPS
- Monitor port: 443
- Monitor path: /api/health
- Custom header: Host:voicetoform.amtccloud.site
- Expected status codes: 200-399,401-401
- Probe interval: 30 seconds; tolerated failures: 3; timeout: 10 seconds.

15.2 Failover proof

With Central enabled, nslookup resolved the custom domain through tm-voicetoform10.trafficmanager.net to 4.174.241.133. During the controlled test, central-agw was disabled in Traffic Manager and the same custom domain resolved to 40.86.207.48. The East application loaded successfully. Central was then re-enabled and DNS returned to 4.174.241.133, proving failback.

15.3 Why HTTP 401 is accepted as healthy

Easy Auth protects the application, so an unauthenticated health request can return HTTP 401 even when the site and network path are healthy. For this architecture, 401 therefore means the endpoint is reachable and authentication is enforcing access. Traffic Manager was configured to accept 401 as a healthy response so it would not incorrectly mark a working authenticated application as down.

16. Monitoring and observability

Monitoring is built into the solution at several layers so the team can see whether the web application, functions, gateways, and backend processing are working.

- Application Insights - requests, dependencies, failures and response performance.
- Log Analytics - central query workspace for collected logs.
- Azure Monitor - metrics, alerts and resource-level monitoring.
- Function Monitor - invocation history and success/failure for QueueWorker and KeepWarm.
- Application Gateway Backend health - verifies connectivity from the gateway to APIM/backend.
- Traffic Manager endpoint status - shows Online/Degraded state for Central and East.

For final evidence, the strongest monitoring screenshots are Application Insights/Live Metrics, a successful Central QueueWorker invocation, healthy Application Gateway backend status, and Traffic Manager showing both endpoints Online when the full DR path is running.

17. Major technical problems and how they were solved

Problem	Resolution
Queue did not exist	QueueWorker initially returned QueueNotFound because the storage queue had not yet been created. Creating voicetoform-queue fixed the trigger path.
Wrong queue name in East	The East web app targeted voiceform-queue instead of voicetoform-queue. Updating the exact name and redeploying fixed QueueNotFound.
403 writing to storage from East	The East App Service identity lacked data-plane permissions. Blob, Queue and Table Data Contributor roles were assigned on stvoicetoformcc.

Wrong project deployed to East App Service	Function package started with func start and failed because Azure Functions Core Tools were not available. Redeploying VOICETOFORM_WebApp restored node server.js startup.
Application Gateway backend unhealthy	Probe received 401 from authenticated backend. Custom probe matching was changed to accept 401 and backend health became Healthy.
Traffic Manager endpoints degraded	Traffic Manager originally probed with incorrect settings. HTTPS 443, /api/health, correct Host header, and accepted status codes were configured.
Custom-domain login changed to azurewebsites.net	Easy Auth did not understand the original public host through the reverse proxy. authV2 forwardProxy was configured to use X-Original-Host.
Initial certificate not trusted / wrong hostname	A trusted certificate for the exact custom hostname was generated and applied to gateway HTTPS listeners.
Azure portal / CLI differences	Some portal fields and Azure CLI commands behaved differently than expected. The project used Cloud Shell and the authV2 extension when the base az webapp auth command lacked the required proxy options.

18. Security architecture

Security was implemented as layered controls rather than a single security feature.

Security layer	Control	Role
Public edge	Application Gateway WAF	Filters and protects inbound web traffic.
Transport	HTTPS/TLS	Encrypts browser-to-gateway and gateway/backend communication.
Identity	Microsoft Entra External ID	Requires patient authentication and supports sign-up/sign-in.
Authorization	Managed Identity + Azure RBAC	Allows apps to access storage without hard-coded account keys.
Secrets/keys	Azure Key Vault	Protects CMK and certificate material.
Data protection	Storage encryption with CMK	Encrypts stored data using a customer-managed key design.
API layer	API Management	Central location for API policy, throttling and monitoring.
Network / platform	VNet, NSG and Azure resource controls	Adds platform/network boundary controls where configured.
Observability	Application Insights / Azure Monitor	Detects failures and unusual behavior.
Resilience	Traffic Manager regional failover	Improves availability during a regional ingress failure.

19. Testing and evidence

The project was not considered complete simply because resources existed. The final validation focused on proving that the user journey and infrastructure behavior worked.

19.1 Functional tests

- Custom domain opens the clinic site over HTTPS.
- Authentication redirects to sign-in and returns to the custom domain.
- Language selection and microphone recording work in the browser.
- POST /api/submit accepts the recording.
- Audio appears in temp-intake.
- Queue message is created in voicetoform-queue.
- QueueWorker invocation succeeds.
- Submission status reaches completed.
- Generated PDF appears in the pdfs container.
- Frontend displays View completed PDF and the document opens.

19.2 Availability tests

- Central Application Gateway direct host-resolution test returns an accepted authentication response.
- East Application Gateway backend health is Healthy when running.
- Traffic Manager shows both endpoints Online when both gateways are available.
- Disabling Central makes the custom domain resolve to the East public IP.
- The East application loads through the same custom domain.
- Re-enabling Central returns DNS to the Priority 1 Central address.

20. Current operating procedure and cost hygiene

The fully redundant design is useful for demonstration and disaster recovery testing, but some Azure networking services can generate noticeable cost while running. The project therefore uses deliberate start/stop procedures when the DR environment is not needed.

20.1 Stopping the East Application Gateway

```
az network application-gateway stop \
  --resource-group rg-voicetoform10-cae \
  --name agw-voicetoform-cae
```

20.2 Starting it again before a failover test

```
az network application-gateway start \
  --resource-group rg-voicetoform10-cae \
  --name agw-voicetoform-cae
```

When the East Application Gateway is stopped, Canada East is not available as an active failover path. Before demonstrating high availability, start the gateway, verify backend health, and wait for Traffic Manager to mark east-agw Online.

Normal project-demo state

For a full resilience demonstration: Central = Priority 1 Online, East = Priority 2 Online. For cost-saving outside the demo: the East gateway may be stopped, but the report should clearly state that DR is temporarily offline while stopped.

21. Lessons learned and future improvements

21.1 Key lessons

- Cloud integrations often fail because of identity, DNS, hostname, or health-probe behavior rather than application code.
- Exact resource names matter. A one-word difference in a queue name can break the full pipeline.
- Managed identity and RBAC are preferable to distributing storage keys.
- Health checks must reflect the actual security model. An authenticated application may legitimately return 401 to anonymous probes.
- Reverse-proxy authentication needs original-host awareness so users stay on the public custom domain.
- Disaster recovery should be tested by actually failing over and failing back, not just by creating duplicate resources.
- Deployment logs are essential for distinguishing application-code failures from Azure configuration failures.

21.2 Recommended future improvements

- Automate infrastructure deployment using Bicep or Terraform so Central and East remain consistent.
- Store source code in GitHub and use GitHub Actions or Azure DevOps for repeatable CI/CD.
- Implement fully automated and monitored TLS certificate renewal.
- Decide and document the production DR data strategy: shared storage versus regional replication and queue ownership.
- Add alert rules for failed submissions, queue backlog, gateway health, certificate expiry, and Traffic Manager endpoint changes.
- Add formal load testing and performance baselines.
- Add structured audit logging suitable for a real healthcare environment and complete any required privacy/compliance assessment before handling real patient data.
- Use synthetic, non-identifiable test data until the solution has undergone formal production security and privacy review.

22. Conclusion

The Voice-to-Form Health Clinic project demonstrates much more than a simple website. It combines frontend development, asynchronous serverless processing, AI services, data storage, identity, encryption, API management, WAF-protected ingress, DNS, TLS, monitoring, and multi-region availability in one Azure solution.

The most important architectural achievement is the separation of responsibilities. The browser focuses on patient interaction, the Web App accepts and tracks submissions, Storage holds durable data, the queue decouples processing, Functions perform background work, Azure AI services handle speech and language, API Management and Application Gateway protect and route the application, Entra External ID controls access, and Traffic Manager provides regional failover.

The project also demonstrates practical cloud engineering troubleshooting. RBAC failures, queue naming errors, authentication redirects, probe status codes, hostname overrides, certificate trust, and incorrect

deployments were diagnosed and corrected. The final result is therefore both a working application and a practical demonstration of Azure architecture, security, operations, and disaster recovery skills.

Appendix A - Important resource names

Item	Value
Primary resource group	rg-voicetoform10-cc
DR resource group	rg-voicetoform10-cae
Central Web App	app-voicetoform10-cc
East Web App	app-voicetoform10-cae
Central Function App	func-voicetoform-cc
East Function App	func-voicetoform-cae
Central APIM	apim-voicetoform-cc
East APIM	apim-voicetoform-cae
Central Application Gateway	agw-voicetoform-cc
East Application Gateway	agw-voicetoform-cae
Central public IP	4.174.241.133
East public IP	40.86.207.48
Storage Account	stvoicetoformcc
Queue	voicetoform-queue
Temporary audio container	temp-intake
PDF container	pdfs
Status table	submissions
Key Vault	kv-voicetoform10-cc
CMK	cmk-blob
Speech	speech-voicetoform-cc
Translator	translate-voicetoform-cc
Traffic Manager	tm-voicetoform10
Custom domain	voicetoform.amtcloud.site
East Log Analytics	law-voicetoform-cae
External ID tenant name	voiceformclinic

Appendix B - Useful validation commands

Check public DNS

```
nslookup voicetoform.amtcloud.site
```

Test Central gateway with custom host

```
curl -k -s -o /dev/null -w "Central HTTP status: %{http_code}\n" \
--resolve voicetoform.amtcloud.site:443:4.174.241.133 \
https://voicetoform.amtcloud.site/api/health
```

Test East gateway with custom host

```
curl -k -s -o /dev/null -w "East HTTP status: %{http_code}\n" \
--resolve voicetoform.amtcloud.site:443:40.86.207.48 \
https://voicetoform.amtcloud.site/api/health
```

Check East Application Gateway state

```
az network application-gateway show \
--resource-group rg-voicetoform10-cae \
--name agw-voicetoform-cae \
--query "operationalState" \
--output tsv
```

Easy Auth original-host configuration

```
az webapp auth update \
--resource-group rg-voicetoform10-cae \
--name app-voicetoform10-cae \
--proxy-convention Custom \
--custom-host-header X-Original-Host
```

Appendix C - Screenshot evidence checklist

- Central resource group resource list.
- East resource group resource list.
- Central and East App Service overview pages.
- Central Function App functions and QueueWorker success monitor.
- Storage Browser showing temp-intake, pdfs, voicetoform-queue, and submissions.
- Completed row in the submissions table.
- Generated PDF blob in the pdfs container.
- Central and East API Management pages.
- Central and East Application Gateway overview pages.
- Application Gateway backend health = Healthy.
- HTTPS listener and trusted certificate configuration.
- Key Vault certificate page without secrets.
- App Service Authentication page showing Microsoft identity provider.
- Traffic Manager endpoints showing Central Priority 1 and East Priority 2.
- nslookup proof for Central resolution.
- nslookup proof for East failover resolution.
- VS Code Web App project structure and server.js API routes.
- VS Code Function project and QueueWorker.js.
- Website welcome screen before recording.
- Voice recording screen.
- Processing screen after submission.
- Completed screen with View completed PDF.
- Generated patient intake PDF.
- Application Insights / Live Metrics.
- Optional Azure Resource Visualizer images.

Final presentation tip

For the strongest presentation, explain the architecture first, then demonstrate one patient submission, show the queue/function completion evidence, and finish by explaining Traffic Manager failover from Canada Central to Canada East.

Appendix D - Short presentation explanation

A concise way to present the project is:

I built a secure Azure Voice-to-Form patient intake application. A patient signs in through Microsoft Entra External ID, selects a language, and records a health statement in the browser. The Node.js App Service saves the audio to Azure Storage and sends a message to Azure Queue Storage. An Azure Function processes the queued submission, uses Azure AI Speech to create a transcript, translates the text to English when required, and produces the final PDF. The status is stored in Azure Table Storage so the frontend can show progress and provide the completed document. In front of the application I use API Management and Application Gateway with WAF and HTTPS. The public domain is routed through Azure Traffic Manager, with Canada Central as Priority 1 and Canada East as Priority 2. I tested both failover and failback, and I also configured Application Insights, Log Analytics, Key Vault, managed identities, RBAC, and customer-managed encryption to demonstrate security, monitoring, and resilience.

Recommended live demo sequence

1. Open the custom domain and show the secure patient login.
2. Record and submit a short test statement.
3. Show the processing state and then the completed PDF.
4. Show the Storage queue/table and a successful QueueWorker execution.
5. Show Application Gateway backend health and Traffic Manager endpoints.
6. Explain the controlled Central-to-East failover test and final failback.